

M.S. Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

Course Name: Artificial Intelligence(AI)

Course Code: AD53

Credits: 3:0:0

UNIT - 5

Term: October 2024 – Jan 2025

Outline

Genetic Algorithms – (Text Book 2, Chapter 9)

- Representing hypotheses,
- Genetic Operators,
- Fitness Function and Selection,
- An Illustrative Example

Robotics- (Text Book 1, Chapter 26)

- Robots, (26.1)
- Robot Hardware, (26.2)
- What kind of problem is robotics solving, (26.3)
- Reinforcement Learning in Robotics, (26.7)
- Application Domains. (26.10)

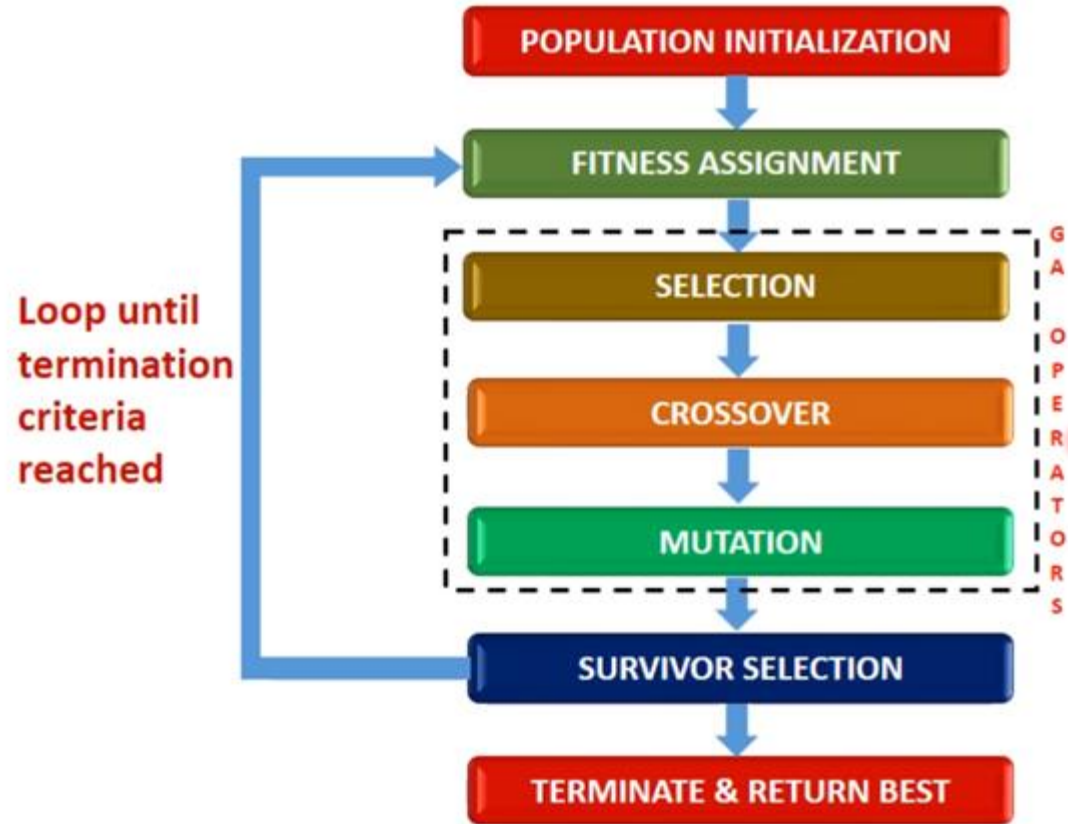
Philosophy, Ethics, and Safety of AI –(Text Book 1, Chapter 27)

- The Limits of AI (27.1)
- Ethics in AI (27.3)
- AI Safety (27.3)

Genetic Algorithm

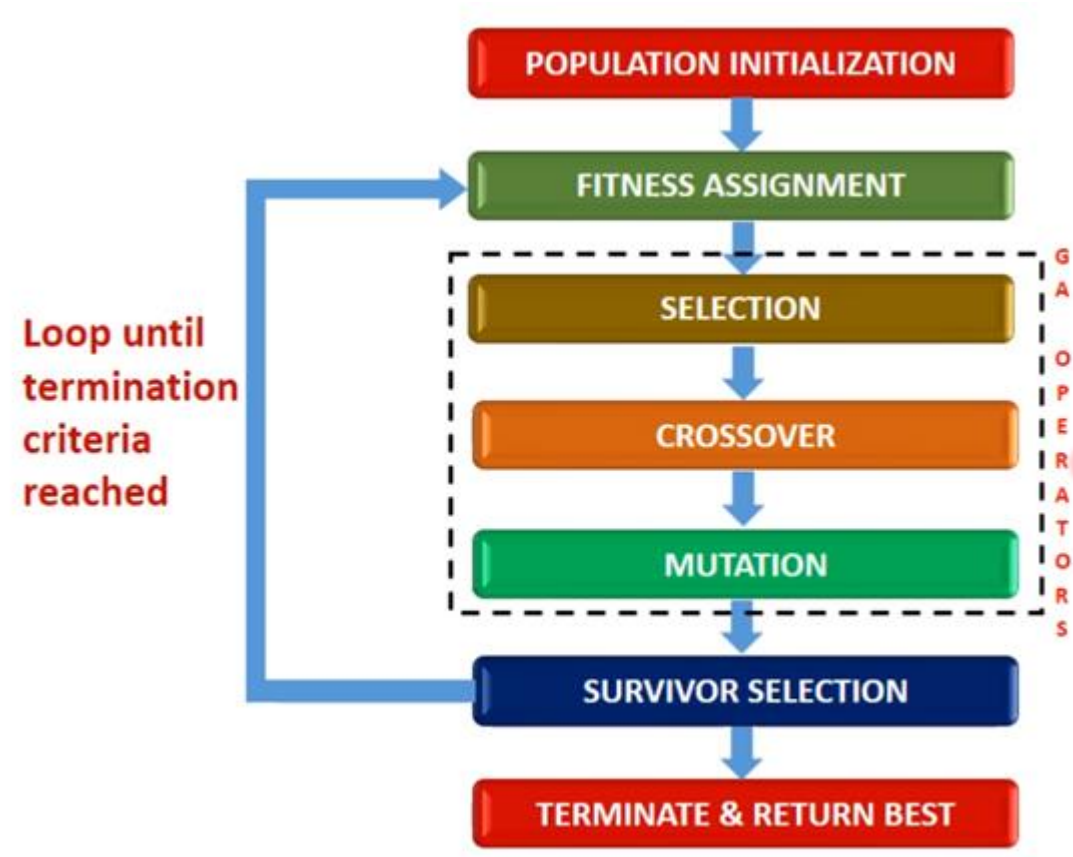
- Evolutionary computing is a subset of artificial intelligence (AI) that uses principles **inspired by biological evolution** to solve complex optimization and search problems.
- It employs **algorithms** that mimic processes such as natural selection, mutation, recombination, and survival of the fittest to iteratively improve solutions to a given problem.
- **Key Concepts in Evolutionary Computing**
- **Population-Based Search:** Evolutionary algorithms work with a population of potential solutions rather than a single solution. This allows for exploration of the search space in parallel.
- **Fitness Evaluation:** Each candidate solution in the population is evaluated using a fitness function that determines how well it solves the problem.
- **Selection:** The most fit solutions are selected to pass their "genes" (characteristics) to the next generation.
- **Crossover (Recombination):** Solutions combine aspects of themselves to create new solutions, simulating reproduction.
- **Mutation:** Random changes are introduced to some solutions to maintain diversity and explore new areas of the search space.

Basic Structure of Genetic Algorithm



- The algorithm operates by iteratively updating a pool of hypotheses, called the **population**.
- On each iteration, all members of the population are evaluated according to the **fitness function**. The fitness of a solution is determined by a fitness function, which quantifies **how "good" or "fit" a solution** is with respect to the problem being solved. In optimization problems, the fitness function could be a **cost function** or a **performance metric**.
- A new population is then generated by **probabilistically selecting the most fit hypothesis** from the current population.

Basic Structure of Genetic Algorithm



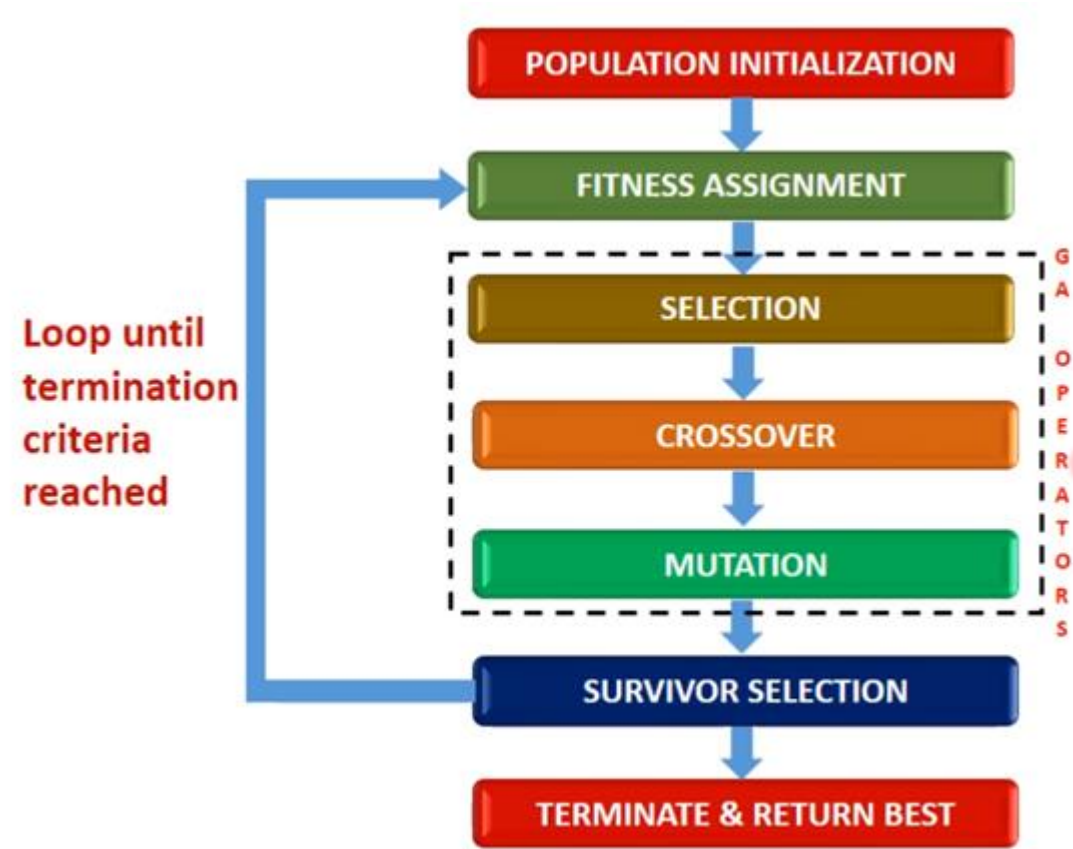
- Some of these **selected hypotheses** are carried forward and are used as the basis for creating new **offspring hypotheses** by applying genetic operations such as **crossover and mutation**.

- **Crossover** simulates the biological process of recombination, where two parent solutions combine their information to produce offspring. The goal is to create new solutions by mixing the characteristics (genetic material) of both solutions.

Example: **Solution 1:** 101010, **Solution 2:** 110110

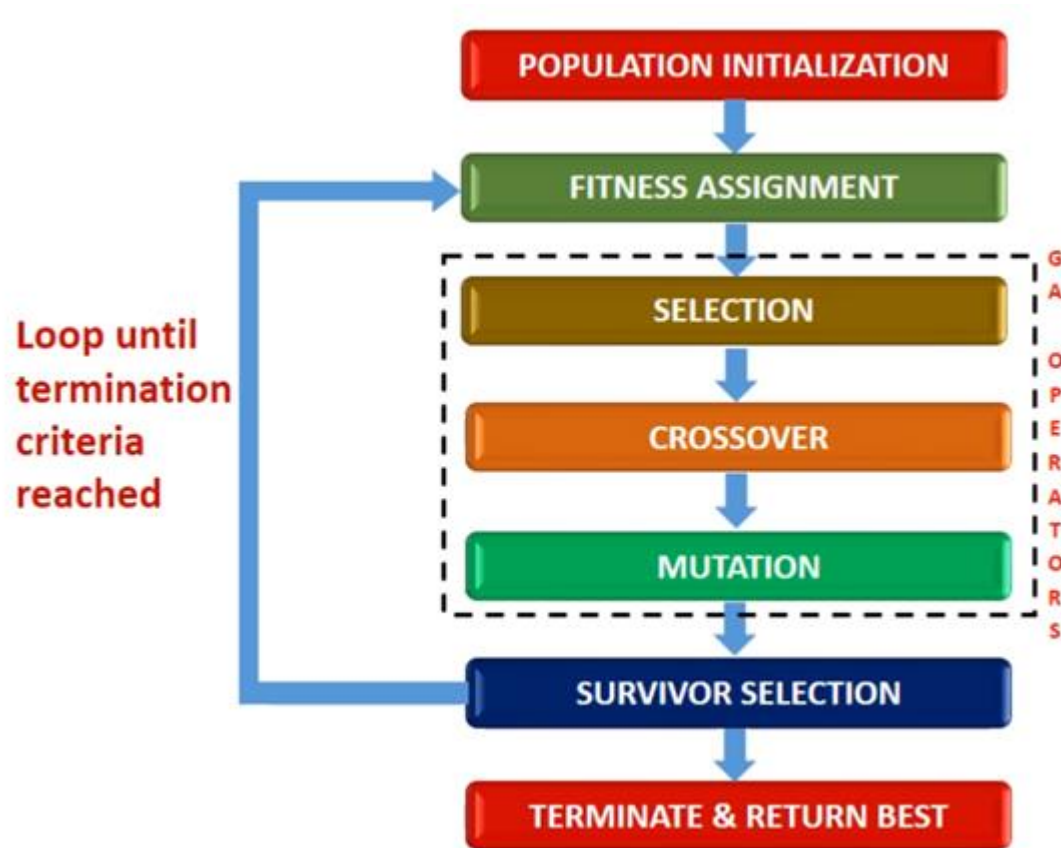
- If the crossover point is 4th bits, the offspring might be:
 - **Offspring 1:** 101110
 - **Offspring 2:** 110010

Basic Structure of Genetic Algorithm



- A **mutation** is applied to an individual solution (usually after the crossover step).
- A random change is made to one or more parts of the solution's. For example, in a binary representation, one bit might be flipped (changing a 0 to a 1, or a 1 to a 0).
- The **mutation rate** is typically small, as too many mutations can disrupt good solutions and slow convergence.
- Example: Solution (before mutation): 101010
- If a mutation is applied to the 4th bit, the offspring might be: Offspring (after mutation): 101110
- Each iteration in the cycle produces a new solutions.

Basic Structure of Genetic Algorithm



- The entire set of generations is called a run.
- Typical GA run is from 50 to 500 or more generations.
- At the end of a run often there is at least one **highly fit solution** in the population.

Representing Hypotheses

- Hypotheses in GA's are often represented by **bit strings**, so that they can be easily manipulated by genetic operators such as mutation and crossover.
- Example, sets of **if-then rules** can be represented, by choosing an encoding of rules that **allocates specific substrings for each rule precondition and postcondition.**

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Representing Hypotheses

Consider Weather Dataset, with single attribute

Attribute = ***Outlook***,

Values ***Sunny, Overcast, or Rain.***

Way to represent a constraint on ***Outlook*** is to use a bit string of length three, in which each bit position corresponds to one of its three possible values.

Ex: 010 → Outlook=Overcast

100 → Outlook=Sunny

001 → Outlook=Rain

011 → ***Outlook = Overcast v Rain***

Representing Hypotheses

- **Consider Conjunction of constraints on multiple attributes can easily be represented by concatenating the corresponding bit strings.**
- Consider a second attribute, ***Wind***, that can take on the value ***Strong*** or ***Weak***. A rule precondition such as

$$(\textit{Outlook} = \textit{Overcast} \vee \textit{Rain}) \wedge (\textit{Wind} = \textit{Strong})$$

can then be represented by the following bit string of length five:

<i>Outlook</i>	<i>Wind</i>
011	10

Representing Hypotheses

- Rule postconditions (such as ***PlayTennis = yes***) can be represented in a similar way.
- The **entire rule** can be described by concatenating the bit strings describing the rule preconditions, together with the bit string describing the rule postcondition. For example, the rule

IF *Wind = Strong* THEN *PlayTennis = yes*

would be represented by the string

<i>Outlook</i>	<i>Wind</i>	<i>PlayTennis</i>
111	10	10

Genetic Operators

- The **generation of successors** in a GA is determined by a set of operators that recombine and mutate selected members of the current population.
- The two most common operators are ***crossover*** and ***mutation***.

Crossover Genetic Operators

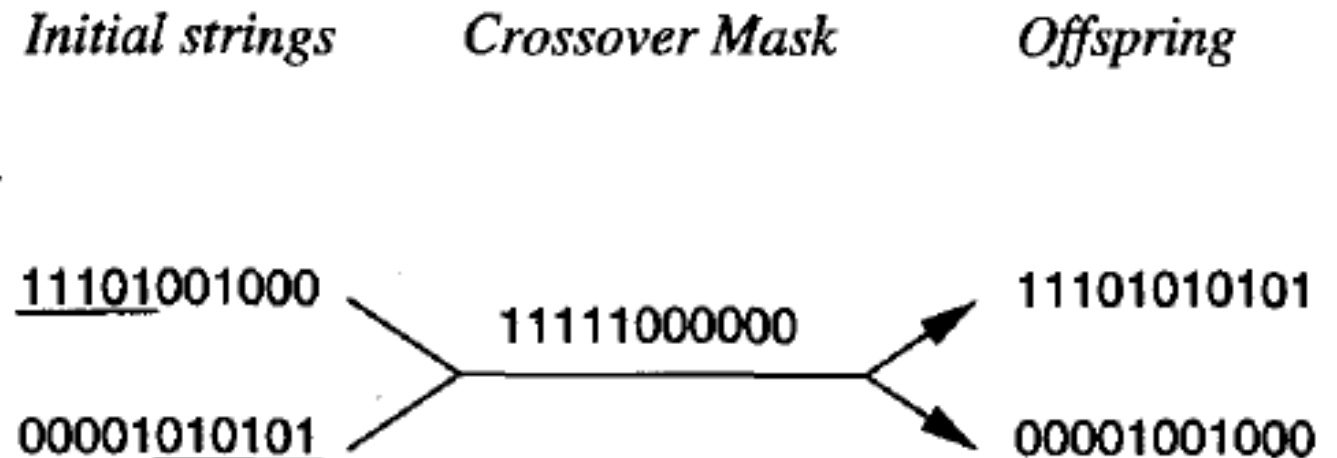
- The ***crossover operator*** produces two new offspring from two parent solution strings, by **copying selected** bits from each parent solution.
- The bit at position ***i*** in each offspring is copied from the bit at position ***i*** in one of the two parent solutions.
- The choice of which parent contributes the bit for position ***i*** is determined by an additional string called the ***crossover mask***.

Crossover Genetic Operators

Single-point crossover

- This **First offspring** takes its **first five bits** from the first parent and its remaining six bits from the second parent
- Crossover mask 11111000000 specifies these choices for each of the bit positions.
- The **second offspring** uses the same crossover mask, but switches the roles of the two parents. Therefore, it contains the bits that were not used by the first offspring.
- In **single-point crossover**, the crossover mask is always constructed so that it begins with a string containing n contiguous 1's, followed by the necessary number of 0s to complete the string.
- This results in offspring in which the **first n bits** are contributed by one parent and the remaining bits by the second parent. Each time the single-point crossover operator is applied the crossover point n is chosen at random, and the cross

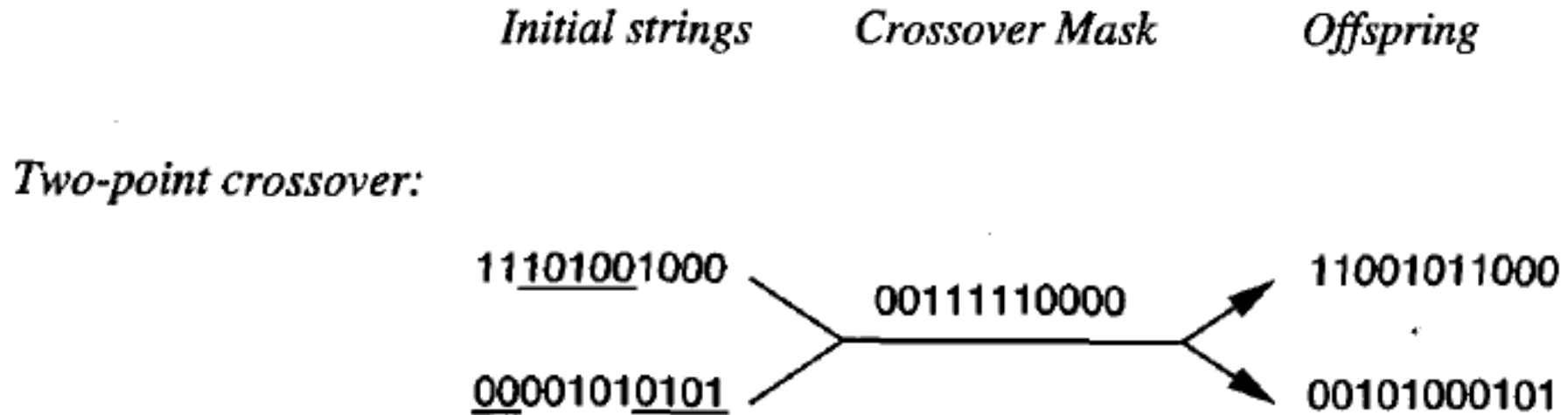
Single-point crossover:



Crossover Genetic Operators

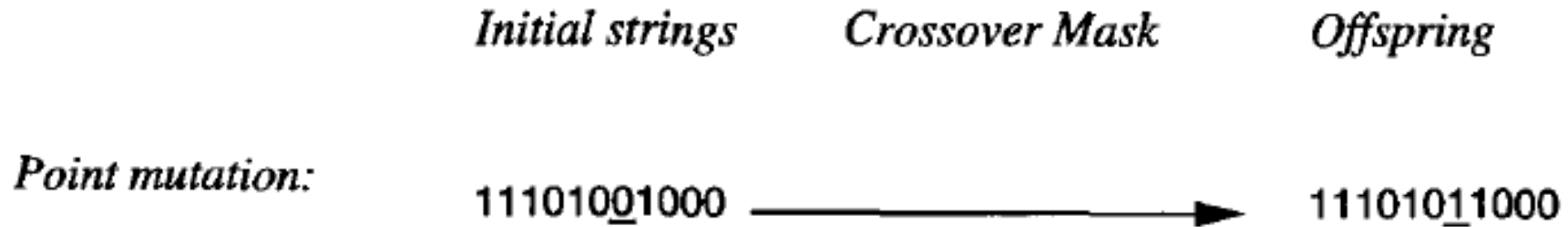
Two-point crossover

- Offspring are created by substituting intermediate segments of one parent into the middle of the second parent string.
- The **crossover mask** is a string beginning with n zeros, followed by a contiguous string of n ones, followed by the necessary number of zeros to complete the string.
- Example : The offspring are created using a mask for which $n0 = 2$ and $n1 = 5$.
- The two offspring are created by switching the roles played by the two parents.



Mutation Genetic Operators

The ***mutation*** operator produces small random changes to the bit string by choosing a single bit at random, then changing its value. Mutation is often performed after crossover.



Fitness Function and Selection

- The fitness function defines the criterion for **ranking potential hypotheses** and for probabilistically selecting them for inclusion in the next generation population.
- If the task is to **learn classification rules**, then the fitness function typically has a component that **scores the classification accuracy** of the rule over a set of provided training examples.
- In prototypical GA shown the probability that a hypothesis will be selected is given by the **ratio of its fitness to the fitness of other members** of the current population. This method is sometimes **called Fitness proportionate selection**, or **roulette wheel selection**

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^P Fitness(h_j)}$$

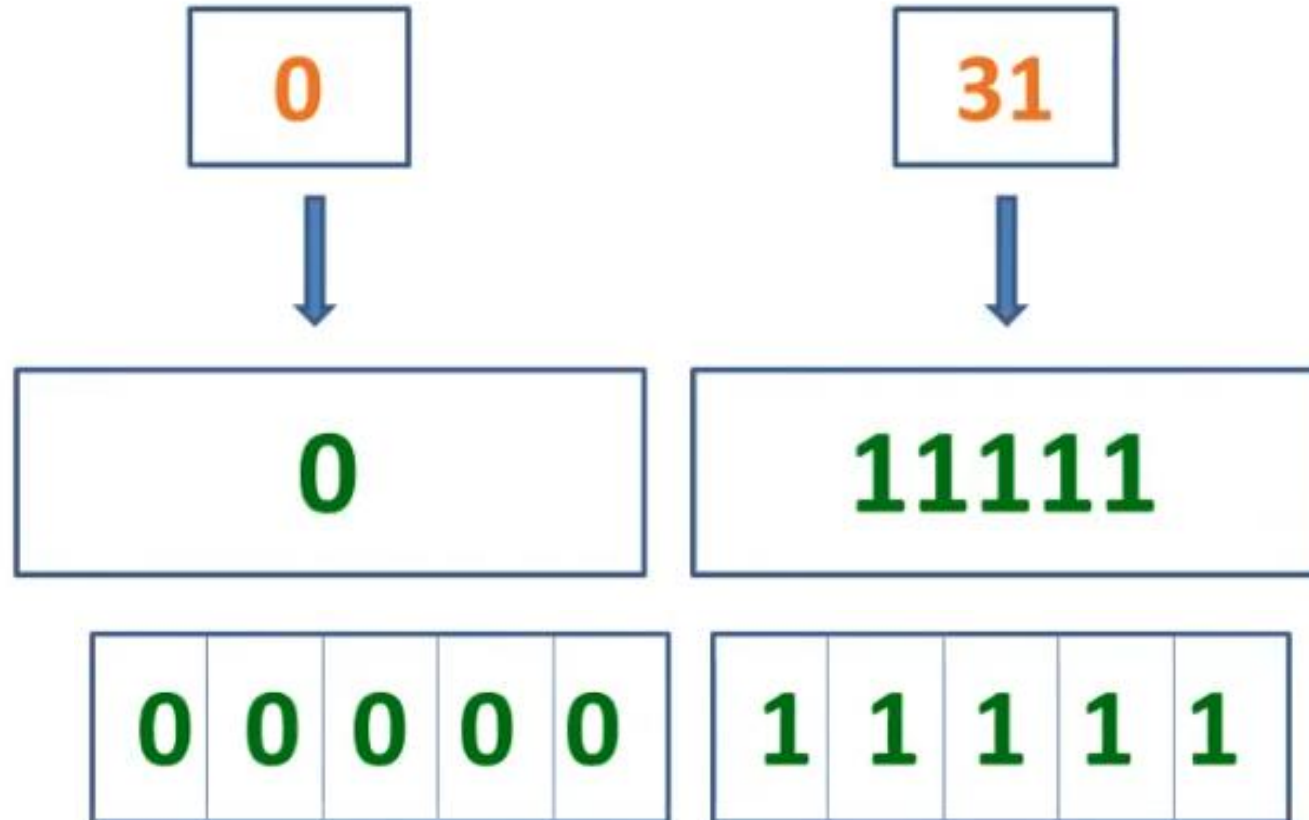
Example

- Optimization Algorithm
- Steps:
 1. Select Encoding type
 2. Choose population size
 3. Randomly choose initial population
 4. Select parental chromosomes
 5. Crossover and mutation
 6. Evaluation of offspring
 7. If stopping criteria not reached, go to step 4

Maximize the function $f(x)=x^2$, where x value range from 0-31

Example

- Step 1: Choose a encoding technique



Example

- Step 2: Choose the population size

$n = 4$

- Step 3: Randomly choose initial population

13, 24 , 8, 19

Example

- Step 4: Select parental chromosomes

String No	Initial Population	X Value	F(x) value	Probability count	Expected count	Actual Count
1		13				
2		24				
3		8				
4		19				

Example

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^P Fitness(h_j)}$$

String No	Initial Population	X Value	F(x) value	Probability count	Expected count	Actual Count
1	01101	13	169	0.14		
2	11000	24	576	0.49		
3	01000	8	64	0.06		
4	10011	19	361	0.31		
Total			1170	1		
Average			293			

Example

String No	Initial Population	X Value	F(x) value	Probability count	Expected count	Actual Count
1	01101	13	169	0.14	=169/293	
2	11000	24	576	0.49		
3	01000	8	64	0.06		
4	10011	19	361	0.31		
Total			1170	1		
Average			293			

String No	Initial Population	X Value	F(x) value	Probability count	Expected count	Actual Count
1	01101	13	169	0.14	0.58	1
2	11000	24	576	0.49	1.97	2
3	01000	8	64	0.06	0.22	0
4	10011	19	361	0.31	1.23	1
Total			1170	1	4	
Average			293			

String 3 is 0 so it is not selected

String 2 is highest fitted chromosome so it is selected twice

String 1 and 4 is selected once

Example

String 3 is 0 so it is not selected
 String 2 is highest fitted chromosome so it is selected twice
 String 1 and 4 is selected once

- Step 5: Crossover and Mutation

String 2	1 1 0 0 0	Parental combination 1
String 1	0 1 1 0 1	
String 2	1 1 0 0 0	Parental combination 2
String 4	1 0 0 1 1	

String 2	1 1 0 0 0	
String 1	0 1 1 0 1	
String 2	1 1 0 0 0	Parental combination 2
String 4	1 0 0 1 1	

String 2	1 1 0 0 0	1 1 0 0 1
String 1	0 1 1 0 1	0 1 1 0 0
String 2	1 1 0 0 0	Crossover point
String 4	1 0 0 1 1	

1 1 0 1 1

1 0 0 0 0

Example

- Step 5: Evaluation of offspring

String No	Offspring 1	X Value	F(x) value
1	0 1 1 0 0		
2	1 1 0 0 1		
3	1 1 0 1 1		
4	1 0 0 0 0		

Example

String No	Offspring 1	X Value	F(x) value
1	0 1 1 0 0	12	144
2	1 1 0 0 1	25	625
3	1 1 0 1 1	27	729
4	1 0 0 0 0	10	256

String No	Offspring 1	X Value	F(x) value
1	0 1 1 0 0	12	144
2	1 1 0 0 1	25	625
3	1 1 0 1 1	27	729
4	1 0 0 0 0	10	256

Mutation

String No.	Offspring after Crossover		Offspring after Mutation at 4 th bit position	X Value	F(x) Value
1	01100		11110	30	900
2	11001		11011	27	729
3	11011		11001	25	625
4	10000		10010	18	324

Original	Crossover	Mutation
576=24	729=27	900=30



GA(*Fitness*, *Fitness_threshold*, *p*, *r*, *m*)

Fitness: A function that assigns an evaluation score, given a hypothesis.

Fitness_threshold: A threshold specifying the termination criterion.

p: The number of hypotheses to be included in the population.

r: The fraction of the population to be replaced by Crossover at each step.

m: The mutation rate.

- *Initialize population*: $P \leftarrow$ Generate p hypotheses at random
- *Evaluate*: For each h in P , compute $Fitness(h)$
- While $[\max_h Fitness(h)] < Fitness_threshold$ do

Create a new generation, P_S :

1. *Select*: Probabilistically select $(1 - r)p$ members of P to add to P_S . The probability $\Pr(h_i)$ of selecting hypothesis h_i from P is given by

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^p Fitness(h_j)}$$



2. *Crossover*: Probabilistically select $\frac{r \cdot p}{2}$ pairs of hypotheses from P , according to $\text{Pr}(h_i)$ given above. For each pair, $\langle h_1, h_2 \rangle$, produce two offspring by applying the Crossover operator. Add all offspring to P_s .
 3. *Mutate*: Choose m percent of the members of P_s with uniform probability. For each, invert one randomly selected bit in its representation.
 4. *Update*: $P \leftarrow P_s$.
 5. *Evaluate*: for each h in P , compute $\text{Fitness}(h)$
- Return the hypothesis from P that has the highest fitness.
-

TABLE 9.1

A prototypical genetic algorithm. A population containing p hypotheses is maintained. On each iteration, the successor population P_s is formed by probabilistically selecting current hypotheses according to their fitness and by adding new hypotheses. New hypotheses are created by applying a crossover operator to pairs of most fit hypotheses and by creating single point mutations in the resulting generation of hypotheses. This process is iterated until sufficiently fit hypotheses are discovered. Typical crossover and mutation operators are defined in a subsequent table.

Advantages/Importance of Genetic Algorithm

- **Robustness Across Problems**
 - GAs are not problem-specific and can be applied to a **wide range of optimization problems**, whether the problem is **continuous, discrete, or mixed**.
- **Global Search Capability**
 - Unlike traditional optimization techniques that may get trapped in **local optima**, GAs use population-based search strategies, increasing the likelihood of finding a **global optimum**.
- **Flexibility in Problem Representation**
 - GAs can handle various types of data representations, such as **binary strings, real numbers, or custom encodings** (e.g., sequences for scheduling problems).
- **Efficient Exploration and Exploitation**
 - **Crossover** encourages **exploration** of new solutions by combining existing ones.
 - **Mutation** ensures continued exploration and avoids stagnation.

Applications of Genetic Algorithm

Optimization in Complex Systems

- GAs are effective in solving problems where traditional methods fail, such as:
 - Scheduling (e.g., airline crew scheduling).
 - Routing (e.g., the Traveling Salesman Problem).
 - Design optimization (e.g., structural engineering).

Application in Machine Learning

- GAs are used to optimize neural networks (e.g., hyperparameter tuning) and design efficient learning algorithms.

Multi-Objective Optimization

- GAs can handle multiple conflicting objectives, e.g., minimizing cost while maximizing performance

Applications of Genetic Algorithm

Real-World Problem Solving

- They are widely used in industries for optimization, such as:
 - Finance: Portfolio optimization.
 - Manufacturing: Minimizing waste in production.
 - Energy: Optimal power grid distribution.

Exploration of Unknown Search Spaces

- GAs are valuable in exploring problems where the search space is poorly understood or exceptionally large.

Healthcare: Drug design, genetic sequencing, and personalized treatment planning.

Engineering: Circuit design, material optimization, and structural engineering.

Limitations of Genetic Algorithm

1. **Computationally Expensive:** GAs often require a significant amount of computational time and resources, especially for large population sizes or complex problems.
2. **Risk of Premature Convergence:** GAs may converge too early to a suboptimal solution, especially if diversity within the population is not maintained.
3. **Lack of Guarantee for Global Optimum:** While GAs are good at exploring the search space, they do not guarantee finding the global optimum, particularly in highly complex landscapes.
4. **Sensitive to Parameter Settings:** The performance of GAs heavily depends on the choice of parameters, such as population size, mutation rate, crossover rate, and selection strategy. Poor tuning can lead to inefficiency.
5. **Difficulty in Fitness Function Design:** Developing an appropriate fitness function that accurately represents the problem objectives can be challenging and may require domain expertise.

Limitations of Genetic Algorithm

6. Dependence on Randomness: GAs rely on stochastic processes (e.g., mutation and crossover), which can introduce unpredictability in performance and outcomes.

7. Scalability Issues: As the dimensionality or complexity of the problem increases, the required population size and number of generations can grow exponentially, making GAs impractical for very high-dimensional problems.

Introduction to Robotics

- Robots are physical agents that perform tasks by **manipulating the physical world**.
- To do so, they are equipped with **effectors** such as legs, wheels, joints, and grippers.
- Effectors have a single purpose: **to assert physical forces on the environment**.
- Few **things may happen**: the robot's state might change, the state of the environment might change, and even the state of the people around the robot might change
- Robots are also equipped with **sensors**, which allow them to perceive their environment.
- **Maximizing expected** utility for a robot means choosing how to actuate its effectors to assert the *right* physical forces—the ones that will lead to changes in state that accumulate as much expected reward as possible

Introduction to Robotics

Robots operate in environments that are

- Partially Observable : cameras cannot see around corners, and gears can slip.
- Stochastic : People acting in that same environment are unpredictable, so the robot needs to make predictions about them.
- Continuous state space (the robot's position has continuous coordinates) and a continuous action space (the amount of current a robot sends to its motor is also measured in continuous units).

Types of Robots

Robots can be categorized based on their design, functionality, and applications. Below is an overview of the major types of robots:

1. Manipulators:

- **Manipulators** are just robot arms.
- They do not necessarily have to be attached to a robot body;
- They might simply be bolted onto a table or a floor
- Some have a large payload, like those assembling cars,
- **Wheelchair-mountable** arms that assist people with motor impairments can carry less but are safer in human environments.
- **Common industrial robots.**



Types of Robots

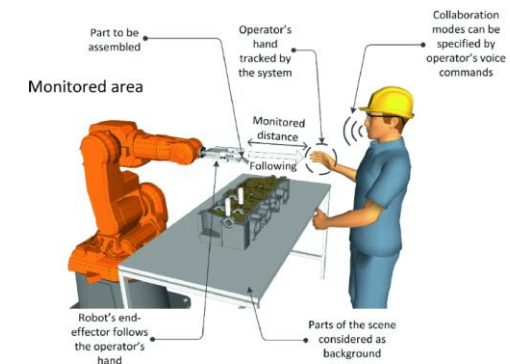
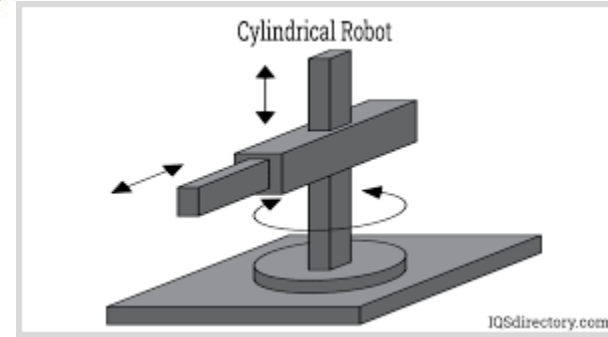
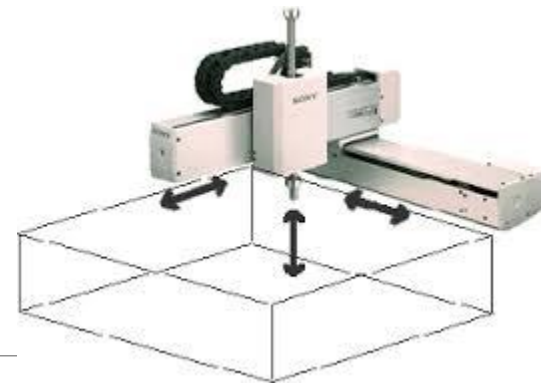
Industrial Robots

Robots designed for manufacturing and industrial automation tasks.

Types:

- **Cartesian Robots:** Operate on three linear axes (X, Y, Z).
- **Cylindrical Robots:** Combine linear and rotary motion.
- **Delta Robots:** Lightweight and fast, used for pick-and-place tasks.
- **Collaborative Robots (Cobots):** Work alongside humans safely.

Applications: Welding, assembly, painting, material handling, and packaging.



Types of Robots

Service Robots

Robots designed to assist humans in non-industrial tasks.

Types:

- **Domestic Robots:** Perform household tasks (e.g., robotic vacuum cleaners).
- **Medical Robots:** Assist in surgeries, diagnostics, and patient care (e.g., Da Vinci surgical system).
- **Entertainment Robots:** Designed for fun and interaction (e.g., robotic pets, animatronics).
- **Educational Robots:** Used for teaching programming and robotics concepts.

Applications: Healthcare, education, and personal assistance.



Types of Robots

Mobile robots are those that use wheels, legs, or rotors to move about the environment.

Quadcopter drones are a type of **unmanned aerial vehicle (UAV)**; **autonomous underwater vehicles (AUVs)** roam the oceans.

Many mobile robots stay indoors and move on wheels, like a **vacuum cleaner** or a towel delivery robot in a hotel.

Legged robots are meant to traverse rough terrain that is inaccessible with wheels. The downside is that controlling legs to do the right thing is more challenging than spinning wheels.



Types of Robots

Swarm Robots

A group of small robots working together to achieve a task.

Characteristics: Decentralized control and collective intelligence.

Applications: Environmental monitoring, disaster response, and exploration

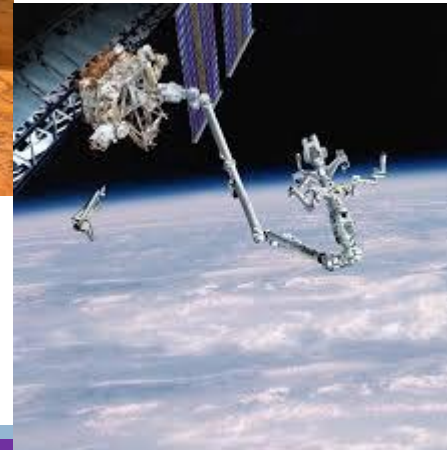
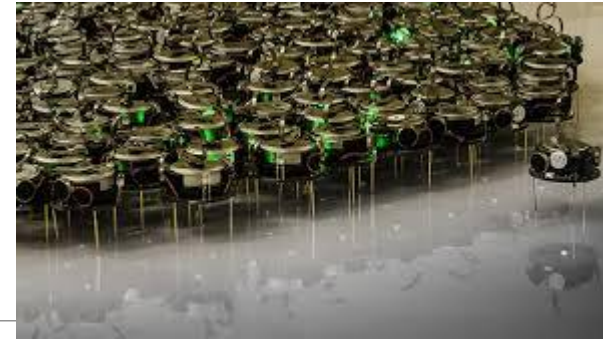
Space Robots

Robots designed for operations in space environments.

Examples: Mars rovers (e.g., Perseverance, Curiosity).

Robotic arms on spacecraft (e.g., Canadarm).

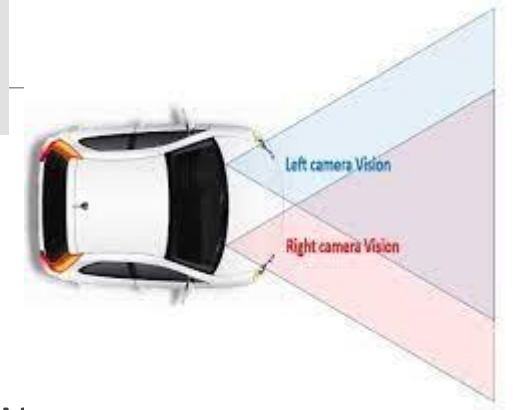
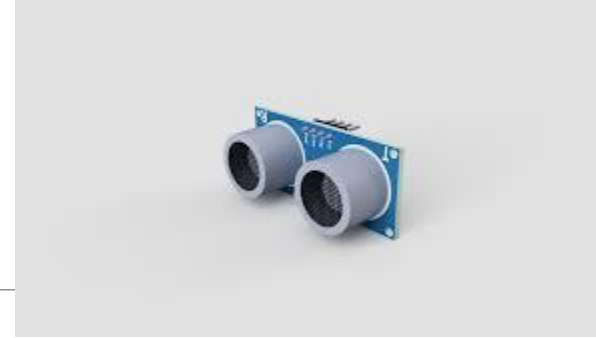
Applications: Space exploration, satellite servicing, and construction.



Robot Hardware: Sensors

- Agent architecture consists of **sensors, effectors, and processors**.
- But the success of real robots depends at least as much on the **design of sensors and effectors that are appropriate for the task**.
- Sensors are the **perceptual interface** between robot and environment.
- **Passive sensors**, such as cameras, are true observers of the environment: they capture signals that are generated by **other sources** in the environment.
- **Active sensors**, such as sonar, **send energy** into the environment.
- They rely on the fact that this **energy is reflected back to the sensor**.
- Active sensors tend to **provide more information** than passive sensors, but at the **expense** of increased power consumption and with a danger of interference then multiple active sensors are used at the same time.

Robot Hardware: Sensors



Range finders:

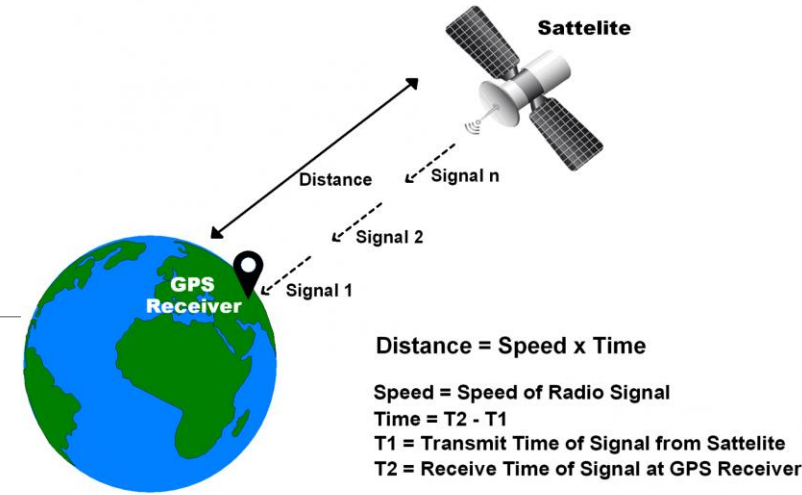
- Sensors that measure the **distance** to nearby objects.
- **Sonar** sensors are active range finders that **emit directional sound** waves, which are reflected by objects, with some of the sound making it back to the sensor. The **time and intensity** of the returning signal indicates the distance to nearby objects.
- Sonar is the technology of choice for autonomous **underwater vehicles**, and was popular in the early days of indoor robotics.
- **Stereo vision** relies on **multiple cameras** to image the environment from slightly different viewpoints, analyzing the resulting parallax in these images to compute the range of surrounding objects

Robot Hardware: Sensors

- **Kinect** is a popular low-cost sensor that combines a camera and a **structured light projector**, which projects a pattern of grid lines onto a scene. The camera sees how the grid lines bend, giving the robot information about the shape of the objects in the scene.



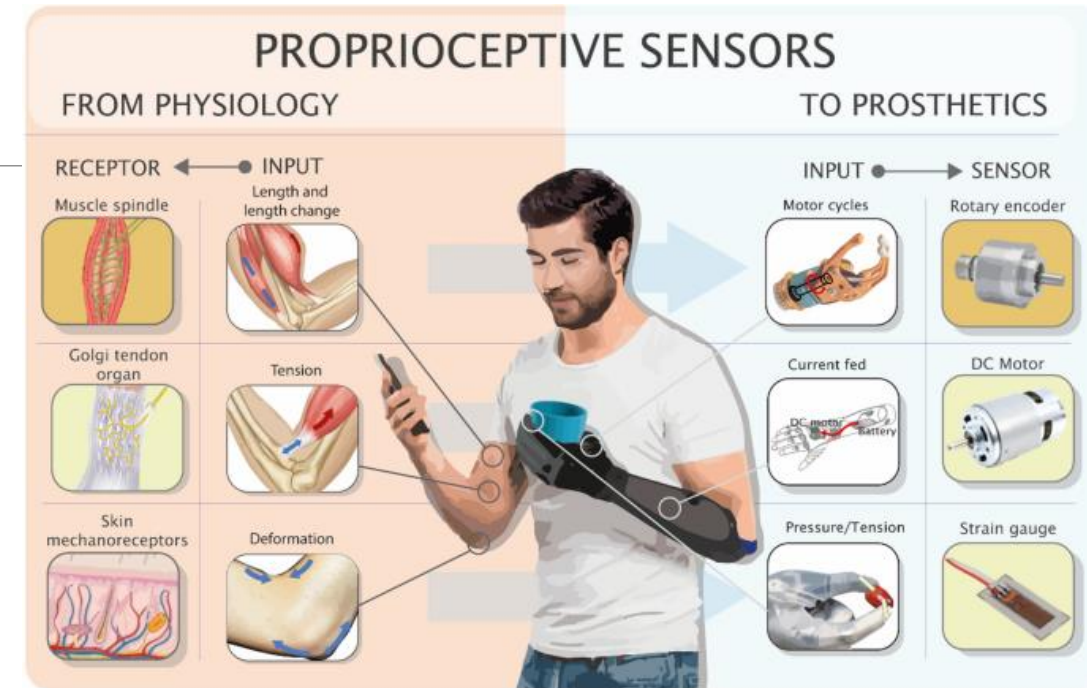
Robot Hardware : Sensor



- **Location sensors:** Primary component to determine location.
- **Global Positioning System (GPS)** is the most common solution to the localization problem. GPS measures the distance to satellites that emit pulsed signals.
- GPS is a system of 30+ navigation satellites circling Earth. We know where they are because they constantly send out signals. A GPS receiver in your phone listens for these signals. Once the receiver calculates its distance from four or more GPS satellites, it can figure out where you are.

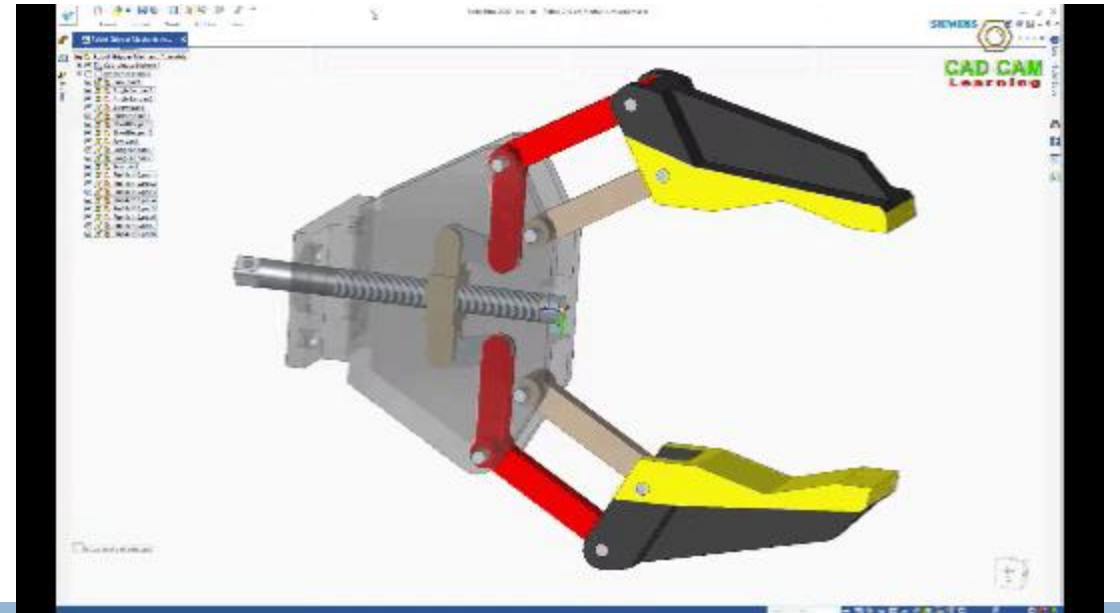
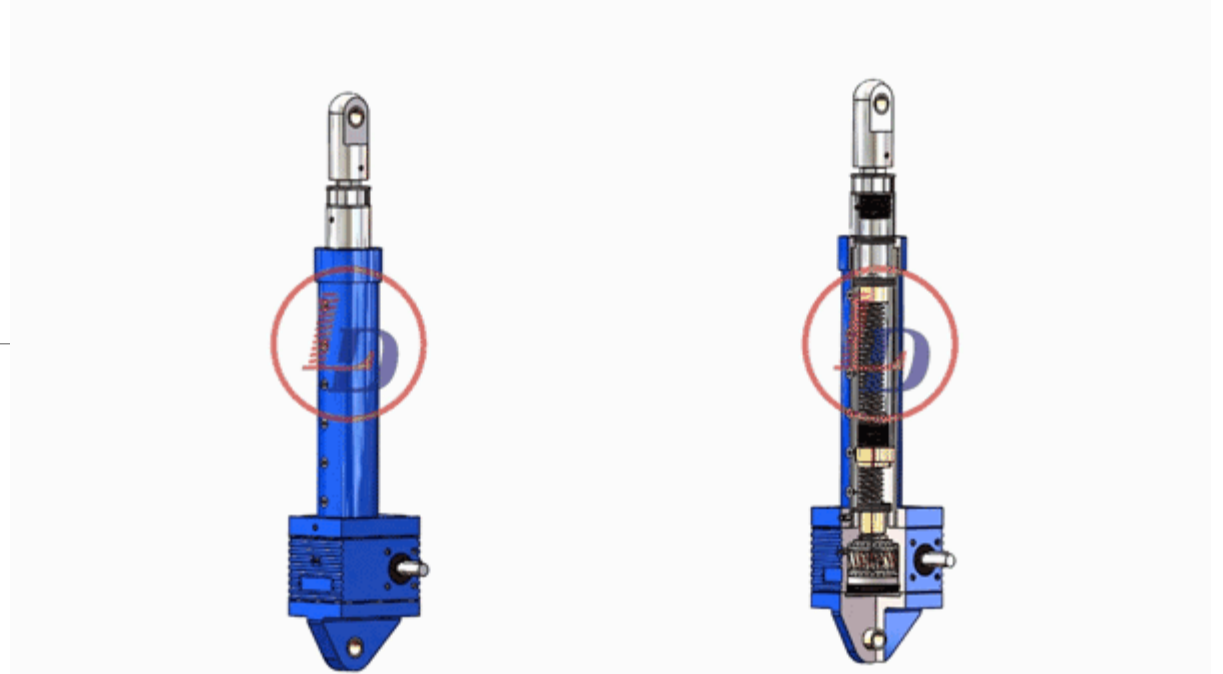
Robot Hardware: Sensors

- The third important class is **proprioceptive sensors**, which inform the robot of its own motion.
- To measure the exact configuration of a robotic joint, motors are often equipped with **shaft decoders** that accurately measure the angular motion of a shaft.
- On mobile robots, shaft decoders report wheel revolutions for **odometry**—the measurement of distance traveled.
- **Inertial sensors**, such as gyroscopes, reduce uncertainty by relying on the resistance of mass to the change of velocity.



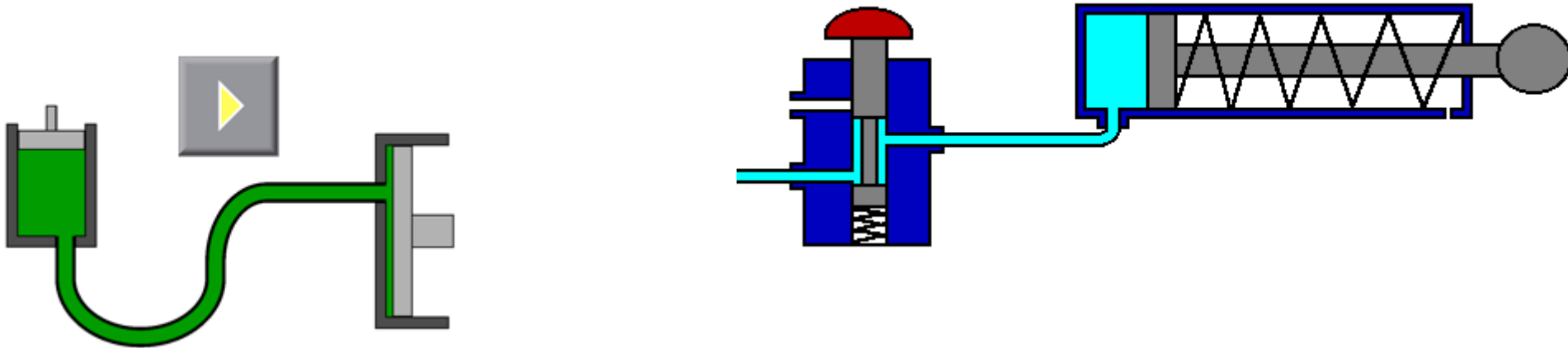
Producing motion/ Actuators

- The mechanism that initiates the motion of an effector is called an **actuator**
- Ex: gears, cables, and linkages.
- **Electric actuator**, uses electricity to spin up a motor. These are predominantly used in systems that need rotational motion, like joints on a robot arm.
- To interact with objects in the environment, robots use **grippers**. The most basic type of gripper is the **parallel jaw gripper**, with two fingers and a single actuator that moves the fingers together to grasp objects.



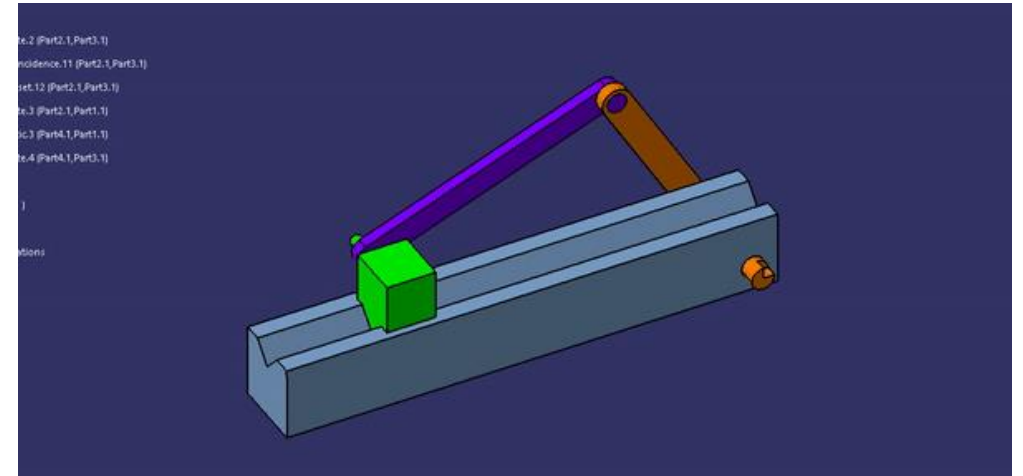
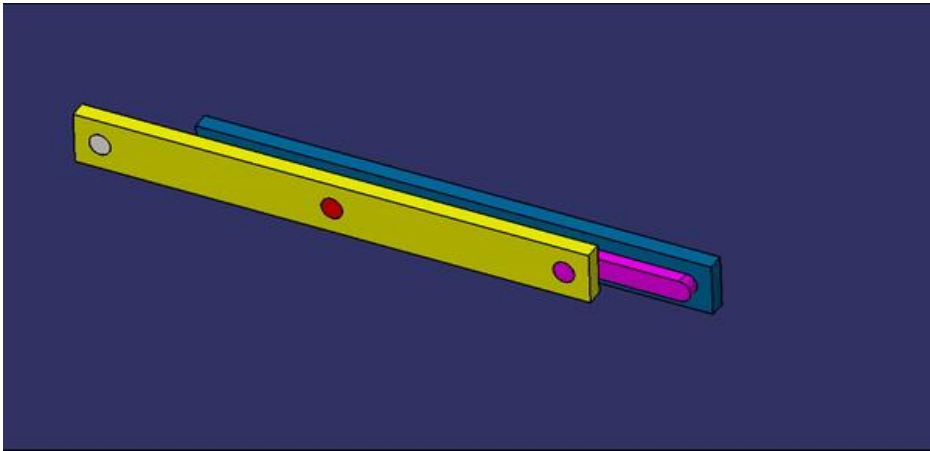
Producing motion/ Actuators

- **Hydraulic actuators** use pressurized hydraulic fluid (like oil or water) and **pneumatic actuators** use compressed air to generate mechanical motion.



Producing motion/ Actuators

Actuators are often used to move joints, which connect rigid bodies (links). Arms and legs have such joints. In **revolute joints**, one link rotates with respect to the other. In **prismatic joints**, one link slides along the other.



What kind of problem is robotics solving?

- The **robot agent software** drives the hardware to achieve the goals.
- There is Need to decide the **computational framework** for the agent.
- Need to **instantiate its ingredients**: reward or utility functions, states, actions, observation spaces, etc. robotics problems are nondeterministic, partially observable, and multiagent.
- Using the game-theoretic notions sometimes the agents are **cooperative** and sometimes they are **competitive**.
- **Example**: In a narrow corridor where only one agent can go first, a robot and a person collaborate because they both want to make sure they don't bump into each other. But in some cases they might compete a bit to reach their destination quickly

What kind of problem is robotics solving?

When robots act in

- **isolation and know their environment**, the problem they are solving can be formulated as an MDP(Markov Decision Process);
- When they are **missing information** it becomes a POMDP(Partially Observable Markov Decision Process) and
- When they act around people it can often be formulated as a **game**.

What is the robot's reward function in this formulation?

Example: Usually the robot is acting in service of a human—for example delivering a meal to a hospital patient for the patient's reward, not its own.

As for the robot's action, state, and observation spaces, the most general form is that **observations** are raw sensor feeds (e.g., the images coming in from cameras, or the laser hits coming in from lidar); **actions** are raw electric currents being sent to the motors; and **state** is what the robot needs to know for its decision making.

What kind of problem is robotics solving?

Robotics often use a three-level hierarchy.

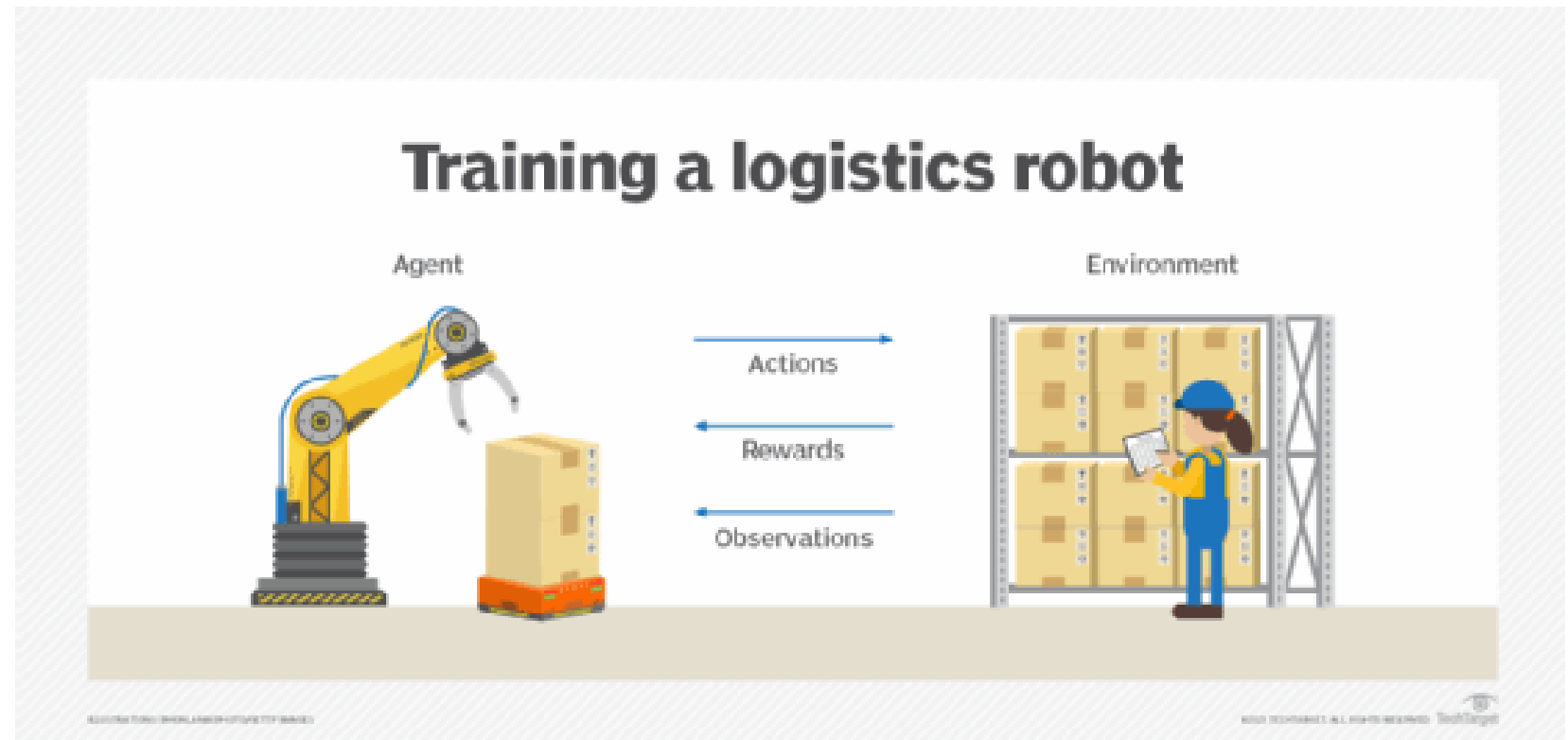
The **task planning** level decides a plan or policy for **high-level actions**, sometimes called action primitives or subgoals: move to the door, open it, go to the elevator, press the button, etc.

Then **motion planning** is in charge of **finding a path** that gets the robot from one point to another, achieving each subgoal.

Finally, **control** is used to achieve the planned motion using the robot's **actuators**.

Reinforcement Learning in Robotics

Reinforcement learning is a machine learning training method based on rewarding desired behaviors and punishing undesired ones. In general, a reinforcement learning agent -- the entity being trained -- is able to perceive and interpret its environment, take actions and learn through trial and error.



Reinforcement Learning in Robotics

- Robot has access to the **dynamic model** of the world.
- In many tasks, it is very **difficult** to write down such a model.

1. Learning from Interaction:

- **Trial and Error:** In RL, a robot learns through interactions with its environment by performing actions and receiving feedback (rewards or penalties). This allows the robot to discover optimal behaviors in complex, dynamic environments.
- **Exploration and Exploitation:** The robot can explore different strategies (exploration) and improve upon previously learned actions (exploitation), which is useful for tasks where exact programming is difficult.

Reinforcement Learning in Robotics

2. Adaptability:

- **Learning in Uncertainty:** RL enables robots to operate in environments with uncertainty or incomplete knowledge. For example, when a robot faces obstacles or changing conditions (e.g., different terrain), it can learn how to adapt its behavior to achieve a goal effectively.
- **Continuous Learning:** Robots can continuously improve their performance over time through reinforcement signals, refining their policies as they gain more experience.

3. Complex Tasks:

- **High-dimensional Control:** Many robotic tasks, such as walking, grasping objects, or navigating, are highly complex and involve continuous control (e.g., joint angles, velocities). RL can optimize these actions without explicitly programming each one.
- **Long-Term Goal Achievement:** RL helps robots learn strategies for long-term goal achievement, where immediate rewards might not be available. This is crucial for tasks like planning, where the consequences of actions unfold over time.

Reinforcement Learning in Robotics : Model Based

- In robotics, model-based reinforcement learning involves the robot learning both the transition dynamics (how actions affect the state of the system) and the reward function (what outcomes lead to positive rewards) of the environment.
- By using this model, the robot can plan ahead and make more informed decisions, often requiring fewer interactions with the environment than traditional model-free RL approaches.

Reinforcement Learning in Robotics : Model Based

Exploiting Model-Based RL in Robotics

Model-based RL in robotics allows for more efficient learning and task execution because it exploits the model of the environment to make predictions about the effects of actions before actually performing them.

1. Learning the Dynamics of the Environment : In model-based RL, the robot learns a model that predicts how its actions will change the state of the environment (transition dynamics). For example, if a robot is learning to move its arm, it may learn how its arm's position changes in response to specific control inputs.

2. Planning and Decision Making : Planning capability allows the robot to solve more complex tasks, such as navigation through an obstacle course or grasping objects in a cluttered environment, where long-term planning is essential.

Reinforcement Learning in Robotics : Model Based

3. Exploration and Exploitation:

In a model-based approach, the robot can balance exploration and exploitation more effectively. Since it has a model of the environment, it can predict outcomes of untried actions and focus on exploring areas where there is uncertainty in the model, rather than performing random exploration. The ability to predict the consequences of exploration can significantly speed up the learning process and improve the overall efficiency of robotic tasks. Example: Model-Based RL for Robotic Arm Control

A classic example of model-based RL in robotics is teaching a robotic arm to manipulate objects. The robot can learn the physics of its own movements and how the environment reacts to them (e.g., how different forces cause objects to move). Using this model, the robot can plan how to pick up an object, move it, and place it in the desired position, all while avoiding obstacles and minimizing energy usage.

Reinforcement Learning in Robotics Model Free

- In Reinforcement Learning (RL), model-free RL is a class of algorithms where the robot **does not explicitly learn a model** of the environment's dynamics (i.e., how the world transitions from one state to another based on actions). Instead, it **directly learns a policy or value function based on interactions with the environment**. Model-free RL is often **simpler and more direct** but may require more interactions with the environment to learn effective behaviors.
- **Policy Optimization:** The robot learns a policy, either through Value-based methods (like Q-learning) or Policy-based methods (like REINFORCE or Proximal Policy Optimization), that dictates what actions to take in each state to maximize long-term rewards.
- **Learning from Rewards:** The robot's goal is to maximize the cumulative reward (or minimize the cost) over time by making better decisions as it interacts with the environment.

Reinforcement Learning in Robotics Model Free

- **Types of Model-Free RL Algorithms**
- **Value-Based Methods:**
 - **Q-Learning:** In Q-learning, the robot learns the value of each state-action pair, essentially storing how good an action is in a particular state. The agent updates the values based on rewards received and chooses actions based on the maximum Q-value.
 - **Deep Q-Networks (DQN):** In more complex environments (such as those with high-dimensional state spaces), DQNs combine Q-learning with deep neural networks to approximate Q-values.
- **Policy-Based Methods:**
 - **REINFORCE:** This is a Monte Carlo method where the robot learns directly the policy (mapping from states to actions) by adjusting it in the direction of greater expected rewards.
- **Actor-Critic Methods:** These combine both value-based and policy-based approaches, where the actor updates the policy, and the critic evaluates how good the action taken by the actor was (usually by learning a value function).

Reinforcement Learning in Robotics Model Free

- **Example: Robot Arm Learning to Pick Up Objects**
- Imagine a robot arm learning to pick up an object from a table. It uses model-free RL to interact with its environment, trying various motions to pick up the object. The robot receives rewards (positive or negative) based on whether it successfully picks up the object or drops it. Over time, it learns which sequences of actions (e.g., the angle of approach, force applied) lead to successful grasps. The robot's policy is refined based on these interactions without needing to learn an explicit model of how its hand interacts with the object.

Reinforcement Learning in Robotics Hybrid Approach

Hybrid approaches borrow ideas from both model-based and model-free algorithms to give us the best of both.

Example: Hybrid RL for Autonomous Navigation

In the case of an autonomous robot navigating an environment (e.g., a robot navigating through a maze):

Model-Based Planning: The robot uses a model of the environment to plan a path that avoids obstacles and optimizes its route. This model might include information about known obstacles, the robot's kinematics, and possible state transitions.

Model-Free Learning: As the robot navigates, it uses a model-free RL algorithm to learn and optimize its policies based on real-world feedback, such as rewards for reaching certain waypoints or penalties for collisions with obstacles. Over time, the robot learns how to optimize its behavior, even in uncharted areas of the environment.

Application Domains

HOME CARE: Robots have started to enter the home to care for **older adults** and people with motor impairments, assisting them with activities of daily living and enabling them to live more independently.

These include wheelchairs and wheelchair-mounted arms like the Kinova arm. Even though they start off as being operated by a human directly, these robots are gaining more and more autonomy. On the horizon are robots operated by **brain-machine interfaces**, which have been shown to enable people with quadriplegia to use a robot arm to grasp objects and even feed themselves.

Figure 26.33



(a) A patient with a brain-machine interface controlling a robot arm to grab a drink. Image courtesy of Brown University. (b) Roomba, the robot vacuum cleaner.

Application Domains

HEALTH CARE: Robots assist and augment surgeons, enabling more precise, minimally invasive, safer procedures with better patient outcomes. The Da Vinci surgical is now widely deployed at hospitals in the U.S.

Figure 26.34



(a)



(b)

(a) Surgical robot in the operating room. Photo by Patrick Landmann/Science Source. (b) Hospital delivery robot. Photo by Wired.

Application Domains

SERVICES: Mobile robots help out in office buildings, hotels, and hospitals. Savioke has put robots in hotels delivering products like towels or toothpaste to your room. The Helpmate and TUG robots carry food and medicine in hospitals, while Diligent Robotics' Moxi robot helps out nurses with back-end logistical responsibilities. Co-Bot roams the halls of Carnegie Mellon University, ready to guide you to someone's office.

Telepresence robots like the Beam to attend meetings and conferences remotely, or check in on our grandparents.



Application Domains

AUTONOMOUS CARS: Some of us are occasionally distracted while driving, by cell phone calls, texts, or other distractions. The sad result: more than a million people die every year in traffic accidents. Further, many of us spend a lot of time driving and would like to recapture some of that time. All this has led to a massive ongoing effort to deploy autonomous cars.

Prototypes have existed since the 1980s, but progress was stimulated by the 2005 DARPA Grand Challenge, an autonomous vehicle race over 200 challenging kilometers of unrehearsed desert terrain.

In 2009, Google started an autonomous driving project (featuring many of the researchers who had worked on Stanley and BOSS), which has now spun off as Waymo. In 2018 Waymo started driverless testing (with nobody in the driver seat) in the suburbs of Phoenix, Arizona. In the meantime, other autonomous driving companies and ride-sharing companies are working on developing their own technology, while car manufacturers have been selling cars with more and more assistive intelligence, such as Tesla's **driver assist**, which is meant for highway driving. Other companies are targeting non-highway driving applications including college campuses and retirement communities. Still other companies are focused on non-passenger applications such as trucking, grocery delivery, and valet parking.

Application Domains

Figure 26.35



(a) Autonomous car Boss which won the DARPA Urban Challenge. Photo by Tangi Quemener/AFP/Getty Images/Newscom. Courtesy of Sebastian Thrun. (b) Aerial view showing the perception and predictions of the Waymo autonomous car (white vehicle with green track). Other vehicles (blue boxes) and pedestrians (orange boxes) are shown with anticipated trajectories. Road/sidewalk boundaries are in yellow. Photo courtesy of Waymo.

Application Domains

EXPLORATION AND HAZARDOUS ENVIRONMENTS: Robots have gone where no human has gone before, including the surface of Mars. Robotic arms assist astronauts in deploying and retrieving satellites and in building the International Space Station. Robots also help explore under the sea. They are routinely used to acquire maps of sunken ships. In 1996, a team of researchers released a legged robot into the crater of an active volcano to acquire data for climate research. Robots are becoming very effective tools for gathering information in domains that are difficult (or dangerous) for people to access.



Application Domains

ENTERTAINMENT: Disney has been using robots (under the name **animatronics**) in their parks since 1963. Originally, these robots were restricted to hand-designed, open-loop, unvarying motion (and speech), but since 2009 a version called **autonomatronics** can generate autonomous actions. Robots also take the form of intelligent toys for children; for example, Anki's Cozmo plays games with children and may pound the table with frustration when it loses.



Application Domains

INDUSTRY: The majority of robots today are deployed in factories, automating tasks that are difficult, dangerous, or dull for humans. (The majority of factory robots are in automobile factories.) Automating these tasks is a positive in terms of efficiently producing what society needs. At the same time, it also means displacing some human workers from their jobs. This has important policy and economics implications—the need for retraining and education, the need for a fair division of resources, etc.

The Limits of AI

In 1980, philosopher John Searle introduced a distinction between

- **weak AI**—the idea that machines could act *as if* they were intelligent—and
- **strong AI**—the assertion that machines that do so are *actually* consciously thinking (not just *simulating* thinking).

Over time the definition of strong AI shifted to refer to what is also called “human-level AI” or “general AI”—programs that can solve an arbitrarily wide variety of tasks, including novel ones, and do so as well as a human.

The Limits of AI

1. **Lack of Understanding and Common Sense:** AI systems lack human-like reasoning and common sense. They operate based on patterns and data, which can lead to errors in unfamiliar or nuanced situations.
2. **Data Dependency:** AI requires large, high-quality datasets for training. If the data is biased, incomplete, or unrepresentative, the AI's outputs will reflect those flaws.
3. **Lack of Creativity:** AI can generate content or ideas based on existing patterns but struggles with genuine creativity or producing entirely original thoughts.
4. **Limited Generalization:** AI systems are typically designed for specific tasks (narrow AI) and struggle to generalize their learning across multiple domains.

The Limits of AI

6. High Computational and Energy Costs

Advanced AI models require significant computational power, which translates into high energy consumption and operational costs.

7. Inability to Handle Ambiguity

AI struggles with ambiguous or poorly defined problems. For example, natural language processing models may misinterpret idioms, sarcasm, or cultural nuances.

8. Vulnerability to Bias

AI can amplify and perpetuate biases present in the training data, leading to unfair or discriminatory outcomes.

9. Lack of Emotional Intelligence

AI cannot truly understand or replicate human emotions, which limits its ability to interact meaningfully in emotional or social contexts.

The Limits of AI

10. The argument from disability :

The “argument from disability” makes the claim that “a machine can never do .”

As examples of , Turing lists the following: Be kind, resourceful, beautiful, friendly, have initiative, have a sense of humor, tell right from wrong, make mistakes, enjoy strawberries and cream, learn from experience, use words properly, be the subject of its own thought, have as much diversity of behavior as man, do something really new.

11. The mathematical objection: Turing (1936) and Gödel (1931) proved that certain mathematical questions are in principle unanswerable by particular formal systems.

The Ethics of AI

- Given that AI is a powerful technology, we have a moral obligation to use it well, **to promote the positive aspects and avoid or mitigate the negative ones.**

The positive aspects are.

- For example, AI can save **lives through improved medical diagnosis**, new medical discoveries, better prediction of extreme weather events, and safer driving with driver assistance and (eventually) self-driving technologies.
- Many opportunities **to improve lives. Microsoft's AI for Humanitarian Action** program applies AI to recovering from natural disasters, addressing the needs of children, protecting refugees, and promoting human rights.
- **Google's AI for Social Good program** supports work on rainforest protection, human rights, pollution monitoring, measurement of fossil fuel emissions, crisis counseling, news fact checking, suicide prevention, recycling, and other issues.
- The University of **Chicago's Center for Data Science for Social Good** applies machine learning to problems in criminal justice, economic development, education, public health, energy, and environment.

The Ethics of AI

- AI applications in **crop management** and food production help feed the world.
- **Optimization of business processes** using machine learning will make businesses more productive, increasing wealth and providing more employment.
- **Automation** can replace the tedious and dangerous tasks that many workers face, and free them to concentrate on more interesting aspects.
- **People with disabilities** will benefit from AI-based assistance in seeing, hearing, and mobility.
- **Machine translation** already allows people from different cultures to communicate.

The Ethics of AI

Many new technologies have had unintended **negative side effects**:

- Nuclear fission brought the threat of **global destruction**; the internal combustion engine brought **air pollution**, global warming.
- Other technologies can have negative effects even when used as intended, such as **sarin gas, AR-15 rifles, and telephone solicitation**.
- **Automation** will create wealth, but under current economic conditions much of that wealth will flow to the **owners of the automated systems**, leading to increased **income inequality**. This can be disruptive to a well-functioning society.
- In developing countries, the traditional path to growth through low-cost manufacturing for export may be cut off, as wealthy countries adopt fully automated manufacturing facilities on-shore. Our ethical and governance decisions will **dictate the level of inequality that AI will engender**.

The Ethics of AI

All **scientists and engineers face ethical considerations** of what projects they should or should not take on, and how they can make sure the execution of the project is safe and beneficial.

In **2010, the UK's Engineering and Physical Sciences Research Council held** a meeting to develop a set of Principles of Robotics.

In subsequent years other **government agencies, nonprofit organizations, and companies** created similar sets of principles.

The gist is that every organization that creates AI technology, and everyone in the organization, has a **responsibility to make sure the technology contributes to good, not harm**. The most commonly-cited principles are:

Ensure safety	Establish accountability
Ensure fairness	Uphold human rights and values
Respect privacy	Reflect diversity/inclusion
Promote collaboration	Avoid concentration of power
Provide transparency	Acknowledge legal/policy implications
Limit harmful uses of AI	Contemplate implications for employment

AI Safety

Almost any technology has the potential to **cause harm in the wrong hands, but with AI and robotics, the hands might be operating on their own.**

It would be unethical to distribute an unsafe AI agent.

Agents should **avoid accidents**, be resistant to adversarial attacks and malicious abuse, and in general to cause benefits, not harms.

That is especially true as AI agents are deployed in safety-critical applications, such as driving cars, controlling robots in dangerous factory or construction settings, and making life-or-death medical decisions.

AI Safety

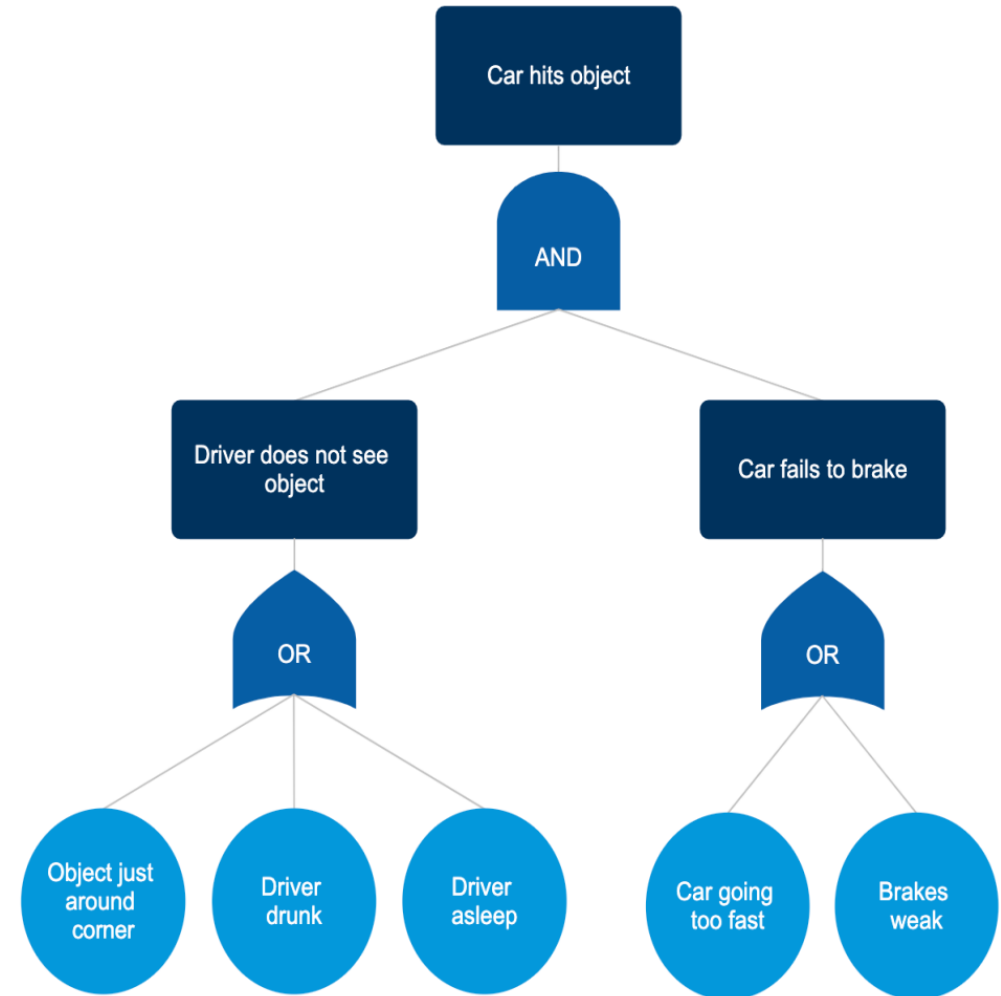
Safety engineering in traditional **engineering fields**. We know how to build bridges, airplanes, spacecraft, and power plants that are designed up front to behave safely even when components of the system fail.

The first technique is **failure modes and effect analysis (FMEA)**: analysts consider each component of the system, and imagine every possible way the component could go wrong (for example, what if this screw were not tightened?), drawing on past experience and on **calculations based on the physical properties of the component**.

Then the analysts work forward to see what would result from the **failure**. If the result is **severe** (a section of the bridge could fall down) then the **analysts alter the design to mitigate the failure**.

AI Safety

The technique of **Fault Tree Analysis (FTA)** is used to make these determinations: analysts build an AND/OR tree of possible failures and assign probabilities to each root cause, allowing for calculations of overall failure probability. These techniques can and should be applied to all safety-critical engineered systems, including AI systems.



AI Safety

The field of **software engineering** is aimed at producing reliable software, but the emphasis has historically been on ***correctness*, not *safety***. Correctness means that the software **faithfully implements the specification**.

Safety goes beyond to insist that the **specification has considered any feasible failure modes, and is designed to degrade gracefully even in the face of unforeseen failures**.

For example, the software for a **self-driving car** wouldn't be considered safe unless it can **handle unusual situations**. For example, what if the **power to the main computer dies**? A safe system will have a **backup computer with a separate power supply**. What if a **tire is punctured at high speed**? A safe system will have tested for this, and will have **software to correct for the resulting loss of control**.